ENGINEERING SYSTEM REFERENCE MANUAL

PriceRadar: Autonomous E-Commerce Price Tracking & Multi-Retailer Monitoring Engine

Complete Technical Workflow, Cloud Architecture, Data Collection Pipeline, Alert Dispatch Engine, and Operational Guide

Project Title:	PriceRadar (Autonomous Price Tracking & Drop Alert System)
Repository:	https://github.com/VamshiYamjala/PriceR (Branch: main)
Production Frontend:	https://priceradar-web.onrender.com
Production Backend API:	https://priceradar-api-02ib.onrender.com
Target Audience:	Engineering Evaluators, Viva Examiners, Development Team, and Maintainers
Inspection Date:	October 2026
Audited Commit:	d845454b543b8713ca174b5f027d6f180d737e45

Abstract: PriceRadar is a full-stack, distributed web platform engineered to eliminate manual price monitoring across major e-commerce retailers (Amazon, Flipkart, Meesho, and eBay). The platform features an asynchronous 4-hour background scraping engine orchestrated via GitHub Actions and Render, an atomic MongoDB distributed locking mechanism preventing race conditions, a deterministic pricing evaluation engine with anti-flapping re-arming rules, and multi-channel instant notifications via Brevo Email and Telegram Bot API.

Table of Contents

Chapter 1	Project Introduction & Executive Abstract
Chapter 2	Complete Technology Stack & Module Mapping
Chapter 3	High-Level Architecture & Subsystem Interactions
Chapter 4	Complete 20-Step End-to-End User & System Workflow
Chapter 5	Frontend Architecture, Component Structure & Routing
Chapter 6	Backend Request Lifecycle, Middleware & Routing Core
Chapter 7	Database Design, Schema Specifications & Entity Relationships
Chapter 8	Product Search, Canonical Matching & Genuine Savings Logic
Chapter 9	Retailer Provider Architecture & Data Extraction Pipeline
Chapter 10	Four-Hour Background Monitoring Engine & Distributed Lock
Chapter 11	Price Observation Ingestion & Alert Evaluation Engine
Chapter 12	Multi-Channel Notification Dispatch (Brevo Email & Telegram Bot)
Chapter 13	Security Architecture, Authentication & Endpoint Authorization
Chapter 14	CI/CD Automation, Test Containers & Cloud Deployment
Chapter 15	Local Development Setup, Tooling & Environment Configuration
Chapter 16	Testing Strategy, Open Handle Resolution & Quality Assurance
Chapter 17	Comprehensive Troubleshooting & Failure Recovery Matrix
Chapter 18	Known System Limitations & Future Architectural Roadmap
Chapter 19	Complete System Quick Reference (Paths, Scripts, Commands)
Chapter 20	Project Demonstration & Examiner Viva Presentation Script

Chapter 1 — Project Introduction & Executive Abstract

1.1 Project Abstract

In modern e-commerce ecosystems, dynamic pricing models cause product prices to fluctuate wildly across platforms such as Amazon, Flipkart, Meesho, and eBay. Consumers face substantial information asymmetry and often miss short-lived price drops. **PriceRadar** solves this challenge by delivering an autonomous, cloud-native price comparison and drop-monitoring system. Built on the MERN stack (MongoDB, Express, React, Node.js) and enhanced with cloud scrapers (Apify), marketplace APIs (eBay Browse API), and standalone Python workers, PriceRadar tracks user-selected listings and checks prices every four hours without requiring client browsers to stay open. Price drops trigger immediate transactional notifications via Brevo Email and Telegram Bot API. The system enforces strict multi-tenant isolation, atomic distributed locking, and integer-based currency precision.

1.2 The Problem Statement

Consumer electronics, household goods, and personal gadgets exhibit extreme price volatility:

- **Fragmented Retailers:** The same smartphone or pair of wireless earbuds may sell for ₹29,999 on Amazon and ₹27,499 on Flipkart during promotional campaigns.
- **Manual Fatigue:** Shoppers cannot continuously refresh multiple retailer tabs across days or weeks.
- **Client Dependency Flaw:** Most consumer price alert tools require browser extensions or open client sessions. If the user shuts down their laptop, monitoring ceases.
- **Misleading Discounts:** Fake "50% off" claims frequently inflate original prices; shoppers lack historical pricing context.

1.3 Motivation and Main Objectives

- **Autonomous Cloud Monitoring:** Decouple the background monitoring cycle from user devices using cloud cron orchestration.
- **Multi-Channel Alerts:** Provide dual-channel notifications (transactional email and real-time mobile Telegram messages).
- **Data Integrity & Sample Safeguards:** Transparently distinguish genuine live retailer data from testbed sample data, preventing simulated listings from triggering real notifications.
- **Deterministic Integer Arithmetic:** Eliminate floating-point rounding errors by representing all monetary amounts in integer Paise (₹1.00 = 100 paise).
- **Scalable Cloud Architecture:** Support container sleep cycles on free-tier cloud PaaS platforms without losing scheduled jobs or corrupting database records.

1.4 Intended Users

1. **Budget-Conscious Shoppers:** Consumers tracking specific electronics waiting for price drops below target thresholds.
2. **Deal Analysts:** Users reviewing historical price trends before major festival sales (Diwali, Big Billion Days).
3. **Software Engineering Evaluators:** Reviewers assessing production-grade distributed architectures, mutex locking, and multi-tier integrations.

1.5 Implemented Features vs. Planned Roadmap

Subsystem	Implemented & Verified in Codebase	Planned Future Enhancements
Authentication	JWT 7-day tokens, bcryptjs (10 rounds), 6-digit email OTP via Brevo, TTL auto-cleanup.	OAuth 2.0 social login (Google, GitHub), password reset flow.
Catalog & Search	MongoDB regex search, canonical key matching, category filters, autocomplete, live fallback.	Elasticsearch full-text indexing, fuzzy spelling correction.
Price Monitoring	4-hour GitHub Actions cron, health wakeup ping, HTTP 202 async trigger, atomic MongoDB lease.	Redis-backed BullMQ horizontally scalable distributed worker queue.
Data Collection	Apify Amazon/Flipkart/Meesho actors, eBay Browse API (OAuth 2.0), fixture dataset.	Headless browser pool with rotating residential residential proxies.
Notifications	Brevo SMTP REST API, Telegram Bot deep linking (`/start `), `deduplication keys.`	WhatsApp notifications via Twilio / Meta Graph API.
Observability	9-category diagnostic runner (A-I), MonitorRun audit logs, PriceObservation history.	OpenTelemetry distributed tracing, Prometheus & Grafana dashboard.

Chapter 2 — Complete Technology Stack & Module Mapping

Every technology documented below is actively verified in the PriceRadar repository.

Technology	Verified Version	Subsystem / Location	Architectural Purpose & Justification
Node.js	`>=18.0.0` (LTS v20 in CI)	Backend Server (`server/`)	Event-driven, asynchronous JavaScript runtime executing the Express API and monitoring tasks.
Express.js	`^4.19.2`	REST API (`server/src/app.js`)	Minimalist HTTP routing framework managing middleware, CORS, JSON parsing, and endpoints.
Mongoose	`^8.5.0`	ODM (`server/src/models/`)	Schema validation, document relationships, compound unique indexes, and MongoDB queries.
MongoDB Atlas	`v7.0` Compatible	Cloud Document DB	High-availability managed NoSQL database storing users, offers, alerts, and execution logs.
React	`^18.3.1`	Client SPA (`client/src/`)	Declarative component-based UI with hooks, virtual DOM reconciliation, and context state.
Vite	`^5.3.4`	Build Engine (`client/vite.config.js`)	Lightning-fast ESM development server and Rollup production asset bundler.
React Router	`^6.24.1`	Navigation (`client/src/App.jsx`)	Client-side SPA declarative route manager with ProtectedRoute guards and ErrorBoundary.
Axios	`^1.7.2` / `^1.20.0`	HTTP Client (Client & Server)	Promise-based HTTP client with request interceptors (JWT injection) and response unwrappers.
jsonwebtoken	`^9.0.2`	Security (`server/src/controllers/authController.js`)	Signs and verifies HMAC SHA-256 tokens for stateless session authorization.
bcryptjs	`^2.4.3`	Security (`server/src/controllers/authController.js`)	One-way salted password hashing (work factor 10) preventing credential leakage.
express-validator	`^7.1.0`	Validation (`server/src/middleware/validate.js`)	Sanitizes and enforces strict validation rules on request bodies, queries, and route parameters.
Apify Client	REST API v2	Data Scraper (`server/src/providers/apify.js`)	Serverless web scraping platform executing actors for Amazon, Flipkart, and Meesho.
eBay Browse API	Buy API v1	Marketplace (`server/src/providers/ebay.js`)	Official eBay developer marketplace API using OAuth 2.0 application client credentials.
FastAPI & Scrapling	`>=0.110.0` / `>=0.2.0`	Microservice (`worker/`)	High-performance Python worker for localized catalog extraction with strict SSRF defenses.
Brevo API	REST API v3	Email Service (`server/src/services/notifications/email.js`)	Transactional SMTP email engine delivering OTP verification codes and price alert emails.
Telegram Bot API	Bot API v6+	Mobile Alerts (`server/src/services/notifications/telegram.js`)	Instant mobile alerts via HTTPS long-polling and deep-linked bot commands.
Jest & Supertest	`^29.7.0` / `^7.0.0`	Testing (`server/tests/`)	Automated integration test runner supporting ES modules and open handle detection.
GitHub Actions	Ubuntu-latest	CI/CD & Cron (`.github/workflows/`)	Executes continuous integration pipeline and the external 4-hour monitoring schedule.
Render	PaaS Platform	Production Host	Decoupled hosting: Backend Web Service + Frontend Static Site with SPA redirect rules.

Chapter 3 — High-Level Architecture & Subsystem Interactions

PriceRadar connects six distinct architectural tiers into a unified, decoupled processing system.

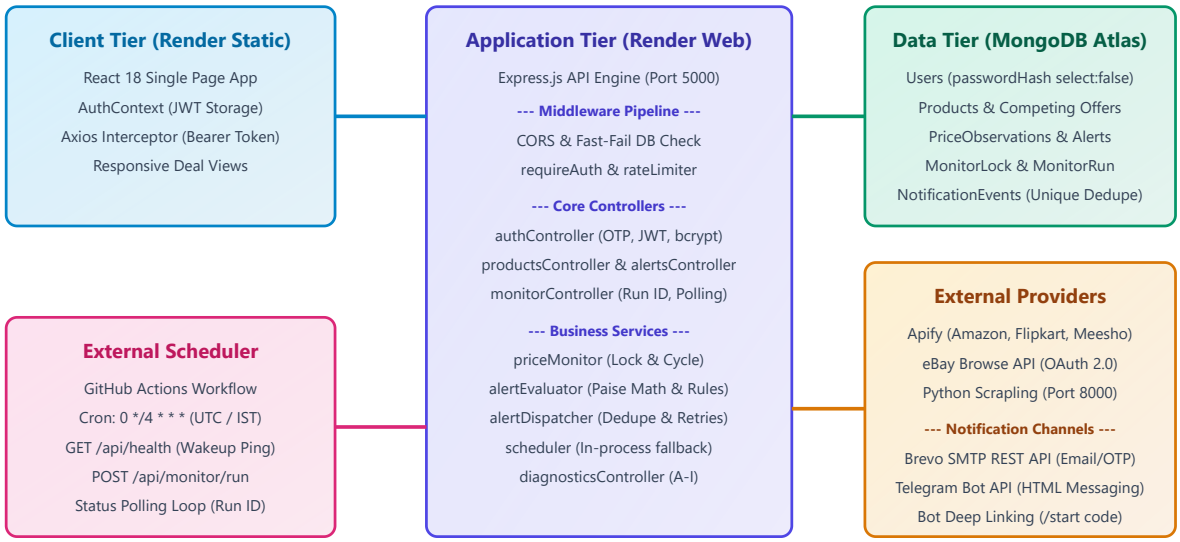


Figure 3.1: Complete PriceRadar System Architecture & Subsystem Communications

3.1 Subsystem Responsibilities

- **Client Tier:** Single-page application rendering fast, reactive views. Injects authorization tokens on protected requests, unwraps `{ success: true, data }` envelopes, and renders responsive pricing tables.
- **External Orchestration Tier:** GitHub Actions triggers the 4-hour cycle from outside Render's sleeping network. Sends health wakeup pings, fires the monitor trigger, and polls the assigned `runId`.
- **Application Tier:** Node.js Express server coordinating routing, database connections, distributed lease acquisition, alert evaluation algorithms, and error formatting.
- **Data Storage Tier:** MongoDB Atlas managed replica set enforcing document constraints, TTL indexes, unique compound indexes, and atomic updates.
- **Data Collection Tier:** Cloud actors (Apify) and marketplace APIs (eBay) extracting live prices, normalized into integer paise by provider adapters.
- **Notification Tier:** Transactional SMTP delivery via Brevo and mobile push messaging via Telegram Bot API with delivery deduplication.

Chapter 4 — Complete 20-Step End-to-End User & System Workflow

The following table traces the complete lifecycle of PriceRadar from initial user visit to autonomous price drop notification:

Step	Workflow Phase	Component / File Path	Exact Function / Route	Technical Action & State Transformation
1	App Visit	`client/src/main.jsx`	`ReactDOM.createRoot()`	Browser loads HTML/JS bundle; initializes Vite client.
2	Session Check	`client/src/context/AuthContext.jsx`	`useState(() => localStorage)`	Derives auth token and user profile synchronously from LocalStorage.
3	Registration Step 1	`client/src/pages/RegisterPage.jsx`	`api.initiateRegistration()`	Submits name, email, password to `POST /api/auth/register/initiate`.
4	OTP Challenge	`server/src/controllers/authController.js`	`initiateRegistration()`	Generates 6-digit OTP, bcrypt-hashes password, sends email via Brevo.
5	OTP Verification	`client/src/pages/RegisterPage.jsx`	`api.verifyRegistration()`	User inputs 6-digit OTP; calls `POST /api/auth/register/verify`.
6	Token Issuance	`server/src/controllers/authController.js`	`verifyRegistration()`	Verifies OTP hash, persists `User` in MongoDB, issues 7-day signed JWT.
7	Product Search	`client/src/pages/SearchPage.jsx`	`api.searchProducts(q, opts)`	Dispatches search query to `GET /api/products/search`.
8	Catalog Query	`server/src/controllers/productsController.js`	`searchProducts()`	Queries MongoDB `Product` catalog via regex; falls back to Apify/eBay.
9	Offer Aggregation	`server/src/models/Offer.js`	`Offer.find({ productId })`	Aggregates competing listings for each matched canonical product.
10	Results Rendering	`client/src/components/ProductCard.jsx`	`ProductCard({ product })`	Renders product cards with lowest price, badges, and retailer count.
11	Product Details	`client/src/pages/ProductDetailPage.jsx`	`api.getProduct(id)`	Renders `OfferRow` list with live/sample badges and savings comparisons.
12	Add to Watchlist	`client/src/components/OfferRow.jsx`	`api.addToWatchlist(offerId)`	Calls `POST /api/watchlist`; persists `WatchlistItem` document.
13	Configure Alert	`client/src/pages/AlertFormPage.jsx`	`api.createAlert(payload)`	Submits target price/drop amount and channels to `POST /api/alerts`.
14	Immediate Evaluation	`server/src/controllers/alertsController.js`	`createAlert()`	If current price $\le$ target price, dispatches alert immediately and disarms.
15	Cron Trigger	`.github/workflows/monitor.yml`	GitHub Actions Cron	Fires every 4 hours; sends wakeup ping to `GET /api/health`.
16	Async Acknowledge	`server/src/controllers/monitorController.js`	`runMonitor()`	Verifies `X-Monitor-Secret`; acquires lock, returns HTTP 202 with `runId`.
17	Offer Scraping	`server/src/services/priceMonitor.js`	`runMonitoringCycle()`	Iterates tracked offers; calls Apify/eBay adapters to fetch prices.
18	Observation Storage	`server/src/models/PriceObservation.js`	`PriceObservation.create()`	Persists price observation; updates `Offer.priceInPaise` and `lastStatus`.

19	Alert Dispatch	`server/src/services/alertDispatcher.js`	`dispatchAlertNotifications()`	Sends Brevo email & Telegram alert; saves `NotificationEvent` dedupe key.
20	Status Polling	`.github/workflows/monitor.yml`	`GET /api/monitor/runs/:id`	Polls run status until complete; fails workflow if all offers failed.

Chapter 5 — Frontend Workflow, Component Structure & Routing

5.1 React 18 Application Bootstrap

The frontend application initializes in `client/src/main.jsx`, mounting the root component into `index.html` inside `BrowserRouter` and `AuthProvider`:

```
// client/src/main.jsx
import React from 'react';
import ReactDOM from 'react-dom/client';
import { BrowserRouter } from 'react-router-dom';
import App from './App';
import { AuthProvider } from './context/AuthContext';
import './styles/theme.css';

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <BrowserRouter>
      <AuthProvider>
        <App />
      </AuthProvider>
    </BrowserRouter>
  </React.StrictMode>
);
```

5.2 Frontend Route Manifest

All application routes are centralized in `client/src/App.jsx` wrapped by a top-level `ErrorBoundary`:

URL Route	Page Component	Access Control	Functional Responsibility
`/`	`HomePage`	Public	Hero section, deal highlights, search input bar, category cards.
`/search`	`SearchPage`	Public	Search filters, retailer filters, price sorting, product grid.
`/products/:productId`	`ProductDetailPage`	Public	Detailed product view, competing offers list, historical price chart.
`/register`	`RegisterPage`	Public (Guest)	Step 1 account registration; Step 2 OTP verification modal.
`/login`	`LoginPage`	Public (Guest)	Email and password login form with validation and error alerts.
`/dashboard`	`DashboardPage`	Protected	Summary of tracked items, active alerts, quick monitor trigger.
`/watchlist`	`WatchlistPage`	Protected	Manage saved retailer offers; delete items; direct link to store.
`/alerts`	`AlertManagementPage`	Protected	List of user alerts, status toggling (active/paused), delete button.
`/alerts/new`	`AlertFormPage`	Protected	Create price alert form: target price or fixed drop threshold.
`/alerts/:alertId/edit`	`AlertFormPage`	Protected	Edit existing alert thresholds, status, or notification channels.
`/settings/notifications`	`NotificationPreferencesPage`	Protected	Email preferences, Telegram linking code generator and instructions.
`/notifications`	`NotificationHistoryPage`	Protected	Audit log of triggered alerts across email and Telegram channels.
`/diagnostics`	`DiagnosticsPage`	Protected / Admin	9-category system health inspection dashboard.
`*`	`NotFoundPage`	Public	HTTP 404 recovery page with navigation back to search.

5.3 State Management & Axios Interceptor

Authentication state is stored in `client/src/context/AuthContext.jsx` using LocalStorage. The singleton API service in `client/src/services/api.js` attaches stored tokens to outgoing HTTP requests and automatically extracts response payloads:

```
// client/src/services/api.js - Request Interceptor & Response Unwrapper
api.interceptors.request.use((config) => {
  const token = localStorage.getItem('priceradar_token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

function unwrap(responsePromise) {
  return responsePromise.then((res) => res.data.data);
}
```

Chapter 6 — Backend Request Lifecycle & Middleware Architecture

6.1 Application Startup & Middleware Sequence

The Express application starts in `server/src/server.js` and mounts middleware in `server/src/app.js` in strict logical order:

1. **Environment Loader:** `server/src/config/env.js` loads environment variables from `server/.env`.
2. **CORS Policy:** Parses `CLIENT_ORIGIN`; allows requests from the frontend domain and server-to-server requests.
3. **Body Parser:** `express.json()` parses JSON payloads.
4. **Health Check Probe:** `GET /api/health` checks MongoDB connection state and returns HTTP 200 or 503.
5. **Database Fast-Fail Middleware:** If MongoDB is disconnected in non-test mode, requests to `/api/*` fail immediately with HTTP 503 (`DATABASE_DISCONNECTED`) instead of timing out.
6. **Route Mounting:** Routes mounted at `/api/auth`, `/api/products`, `/api/offers`, `/api/watchlist`, `/api/alerts`, `/api/notifications`, `/api/telegram`, `/api/monitor`, and `/api/diagnostics`.
7. **404 Catch-All:** Returns HTTP 404 (`NOT_FOUND`) for unmatched routes.
8. **Centralized Error Handler:** `server/src/middleware/errorHandler.js` formats unhandled errors into standard JSON responses.

6.2 Backend Route Inventory

HTTP Method	Endpoint Route	Auth Required	Controller Handler	Description
`GET`	`/api/health`	None	Inline Handler	Liveness probe; returns database connection state.
`POST`	`/api/auth/register/initiate`	None	`initiateRegistration`	Sends 6-digit OTP to user email via Brevo.
`POST`	`/api/auth/register/verify`	None	`verifyRegistration`	Validates OTP; creates `User`; returns JWT.
`POST`	`/api/auth/register/resend`	None	`resendRegistrationOtp`	Resends OTP if 60s cooldown elapsed.
`POST`	`/api/auth/login`	None	`login`	Validates password via bcrypt; returns JWT.
`GET`	`/api/auth/me`	JWT	`getMe`	Returns authenticated user profile.
`GET`	`/api/products/search`	None	`searchProducts`	Searches products with regex and live fallback.
`GET`	`/api/products/categories`	None	`getCategories`	Returns supported category list.
`GET`	`/api/products/autocomplete`	None	`getAutocomplete`	Fast typeahead suggestions for search queries.
`GET`	`/api/products/:productId`	None	`getProductById`	Returns product details and competing offers.
`GET`	`/api/offers/:offerId/history`	None	`getOfferHistory`	Returns historical price observations.
`GET`	`/api/watchlist`	JWT	`listWatchlist`	Lists user saved offers.
`POST`	`/api/watchlist`	JWT	`addToWatchlist`	Adds offer to user watchlist.
`DELETE`	`/api/watchlist/:id`	JWT	`removeFromWatchlist`	Removes offer from user watchlist.
`GET`	`/api/alerts`	JWT	`listAlerts`	Lists user configured alerts.
`POST`	`/api/alerts`	JWT	`createAlert`	Creates alert; triggers immediately if met.
`PATCH`	`/api/alerts/:alertId`	JWT	`updateAlert`	Updates alert threshold, status, or channels.
`DELETE`	`/api/alerts/:alertId`	JWT	`deleteAlert`	Deletes user price alert.
`POST`	`/api/telegram/link-code`	JWT	Inline Handler	Generates 6-character Telegram linking code.
`POST`	`/api/monitor/run`	Secret	`runMonitor`	Triggers background monitoring cycle (HTTP 202).
`GET`	`/api/monitor/runs/:runId`	Secret/Admin	`getMonitorRunStatus`	Polls outcome of specific monitoring run.
`GET`	`/api/diagnostics`	Secret/Dev	`getAllDiagnostics`	Runs 9-category system health checks.

Chapter 7 — Database Design, Schema Specifications & Entity Relationships

PriceRadar uses MongoDB Atlas managed via Mongoose. All currency fields are stored as integer Paise.

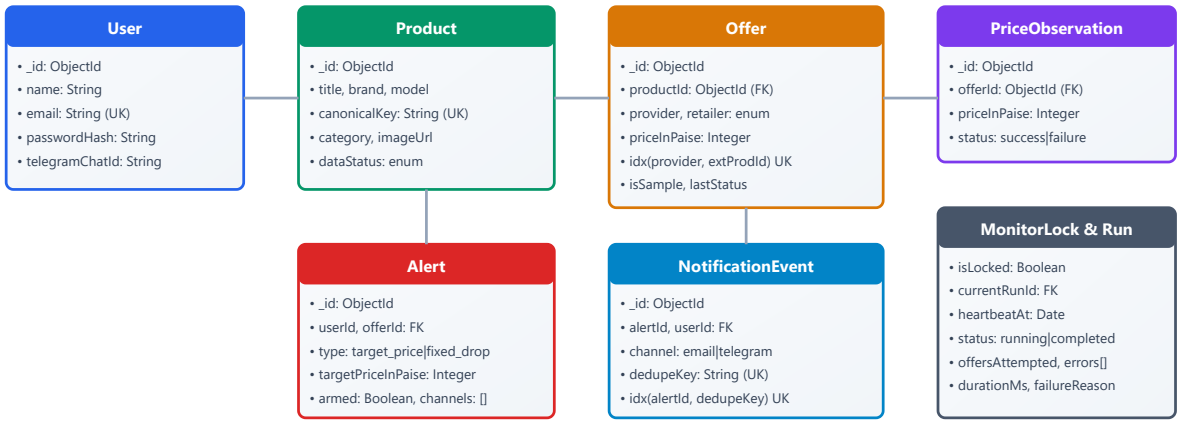


Figure 7.1: Mongoose Database Models, Foreign Keys & Compound Index Architecture

7.1 Critical Indexes & Data Integrity Constraints

- **Duplicate Offer Protection:** `Offer.schema.index({ provider: 1, externalProductId: 1 }, { unique: true })` prevents the same retailer listing from being inserted twice.
- **Price Observation History:** `PriceObservation.schema.index({ offerId: 1, fetchedAt: -1 })` enables sub-millisecond retrieval of historical price charts.
- **Anti-Spam Alert Deduplication:** `NotificationEvent.schema.index({ alertId: 1, dedupeKey: 1 }, { unique: true })` guarantees that an alert cannot deliver duplicate notifications for the same price event.
- **Registration OTP Expiration:** `OtpChallenge.schema.index({ expiresAt: 1 }, { expireAfterSeconds: 0 })` uses MongoDB TTL background thread to auto-delete stale challenges.

Chapter 8 — Product Search, Canonical Matching & Genuine Savings Logic

8.1 Canonical Product Matching Algorithm

Products from multiple sources are normalized into canonical products using `canonicalKey`. For example, search queries for "Sony WH-1000XM5" match across Amazon and Flipkart:

```
// server/src/utils/money.js & productsController.js
// Normalized canonical key: lowercase, alphanumeric and dashes
const canonicalKey = `${brand}-${model}-${variant}`
  .toLowerCase()
  .replace(/[^a-z0-9]+/g, '-')
  .replace(/(^-|-$)/g, '');
```

8.2 Genuine Market Savings vs. Sample Data Safeguards

Implemented in `server/src/services/priceComparison.js`, PriceRadar strictly enforces that simulated sample offers cannot participate in genuine savings calculations:

```
// server/src/services/priceComparison.js
export function calculateGenuineSavings(baseOffer, comparedOffer) {
  if (!baseOffer || !comparedOffer) {
    return { savingsInPaise: 0, savingsPercent: 0, isGenuine: false, reason: 'Missing offer' };
  }
  const baseCurrency = (baseOffer.currency || 'INR').toUpperCase();
  const compCurrency = (comparedOffer.currency || 'INR').toUpperCase();
  if (baseCurrency !== compCurrency) {
    return { savingsInPaise: 0, savingsPercent: 0, isGenuine: false, reason: `Currency mismatch: ${baseCurrency} vs ${compCurrency}` };
  }
  const hasSampleData = baseOffer.isSample || comparedOffer.isSample ||
                        baseOffer.dataStatus === 'sample' || comparedOffer.dataStatus === 'sample';
  const diff = baseOffer.priceInPaise - comparedOffer.priceInPaise;
  const percent = baseOffer.priceInPaise > 0 ? (diff / baseOffer.priceInPaise) * 100 : 0;
  const rawSavings = { savingsInPaise: Math.max(0, diff), savingsPercent: Math.max(0, Number(percent.toFixed(1))) };

  if (hasSampleData) {
    return { ...rawSavings, isGenuine: false, reason: 'Contains sample or simulated catalog data' };
  }
  return { ...rawSavings, isGenuine: true };
}
```

8.3 Worked Price Calculation Example

Listing Offer	Retailer	Price in Paise	Price in Rupees	Availability	isSample	Evaluated Outcome
Listing A	Amazon (Live)	2,999,000	₹29,990.00	in_stock	false	Eligible comparison baseline.
Listing B	Flipkart (Live)	2,699,000	₹26,990.00	in_stock	false	**Lowest valid offer.** Genuine Savings: ₹3,000 (10.0%).
Listing C	Croma (Sample)	2,499,000	₹24,990.00	in_stock	true	Excluded from genuine savings (marked "Sample Data").
Listing D	Reliance (Live)	2,599,000	₹25,990.00	out_of_stock	false	Excluded (out_of_stock offers cannot be lowest offer).

Chapter 9 — Retailer Provider Architecture & Data Extraction Pipeline

9.1 Provider Registry & Dispatcher

Located in `server/src/providers/index.js`, the dispatcher routes requests based on `offer.provider`:

```
// server/src/providers/index.js
export async function fetchOffer(offer) {
  let result;
  if (offer.provider === 'apify') {
    result = await apifyProvider.fetchOffer(offer);
  } else if (offer.provider === 'ebay') {
    result = await ebayProvider.fetchOffer(offer);
  } else if (offer.isSample || offer.provider === 'fixture') {
    result = await fixtureProvider.fetchOffer(offer);
  } else if (offer.provider === 'scrapling') {
    result = await scraplingProvider.fetchOffer(offer);
  } else {
    result = { provider: offer.provider, success: false, errorMessage: `No provider configured for "${offer.provider}"` };
  }
  return { ...result, fetchedAt: new Date().toISOString() };
}
```

9.2 Provider Sequence Diagram

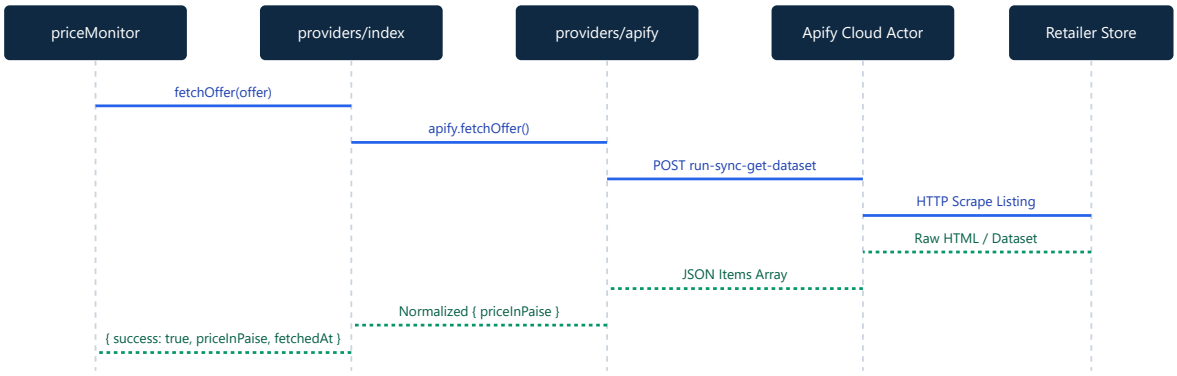


Figure 9.1: Retailer Provider Dispatch and Data Extraction Sequence

Chapter 10 — Four-Hour Background Monitoring Engine & Distributed Lock

10.1 Why Periodic External Scheduling Is Required

Free-tier cloud containers (such as Render Web Services) spin down after 15 minutes of inactivity. When a container sleeps, internal timers (like `setInterval`) stop executing. To maintain an automatic 4-hour monitoring cycle, PriceRadar relies on **GitHub Actions** as an external scheduler. Every 4 hours, GitHub Actions wakes the container and initiates the scrape.

10.2 Cron Schedule: UTC vs. India Standard Time (IST)

Configured in `.github/workflows/monitor.yml` with cron expression `0 */4 * * *`:

Execution Cycle	Universal Coordinated Time (UTC)	India Standard Time (IST = UTC + 5:30)
Cycle 1	00:00 UTC	05:30 AM IST
Cycle 2	04:00 UTC	09:30 AM IST
Cycle 3	08:00 UTC	01:30 PM IST
Cycle 4	12:00 UTC	05:30 PM IST
Cycle 5	16:00 UTC	09:30 PM IST
Cycle 6	20:00 UTC	01:30 AM IST (Next Day)

10.3 The Atomic Distributed Lock (`MonitorLock`)

In multi-instance cloud deployments or simultaneous requests, race conditions can trigger overlapping scrapes. In `server/src/services/priceMonitor.js`, PriceRadar uses an atomic MongoDB distributed document lease:

```
// server/src/services/priceMonitor.js - Atomic Lock Acquisition
export async function acquireDistributedLock(instanceId, staleTimeoutMs = 15 * 60 * 1000) {
  const now = new Date();
  const staleCutoff = new Date(now.getTime() - staleTimeoutMs);

  let lock = await MonitorLock.findOneAndUpdate(
    {
      _id: 'monitoring_cycle_lock',
      $or: [
        { isLocked: false },
        { isLocked: null },
        { heartbeatAt: { $lt: staleCutoff } } // Stale lease recovery
      ],
    },
    { $set: { isLocked: true, acquiredAt: now, heartbeatAt: now, instanceId } },
    { new: true }
  );
  if (lock) return { acquired: true, lock };
  return { acquired: false, reason: 'ACTIVE_LEASE_HELD' };
}
```

10.4 Four-Hour Monitoring Execution Flowchart

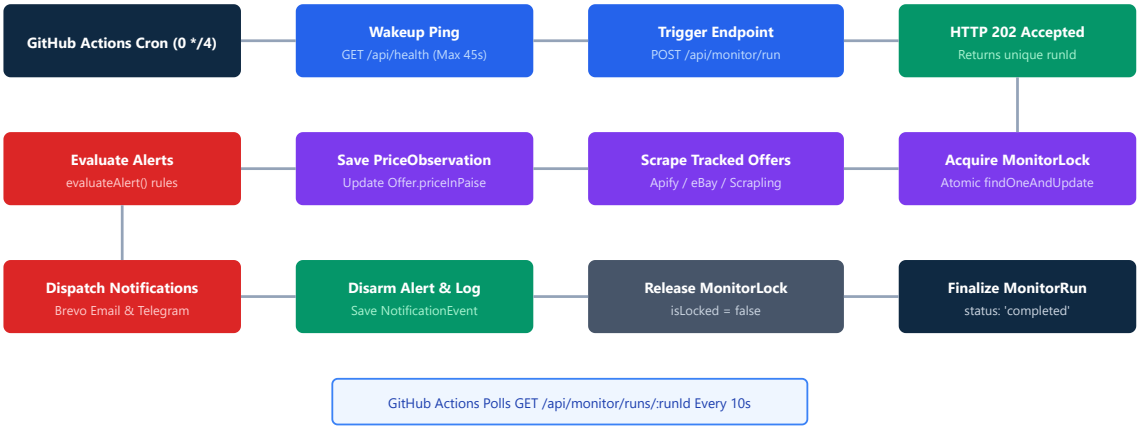


Figure 10.1: Four-Hour Background Monitoring Execution and Status Polling Flowchart

Chapter 11 — Price Observation Ingestion & Alert Evaluation Engine

11.1 The Alert Evaluation Rules

Alert evaluation is implemented in `server/src/services/alertEvaluator.js`. The function strictly rejects zero, negative, NaN, or cross-currency price observations:

```
// server/src/services/alertEvaluator.js
export function evaluateAlert(alert, currentPriceInPaise, previousPriceInPaise = null, options = {}) {
  const alertId = alert?._id ? alert._id.toString() : 'unknown';

  if (typeof currentPriceInPaise !== 'number' || isNaN(currentPriceInPaise) || currentPriceInPaise <= 0) {
    return { conditionMet: false, shouldNotify: false, shouldRearm: false, reason: 'Invalid or non-positive price' };
  }

  let conditionMet = false;
  let reason = '';

  if (alert.type === 'target_price') {
    if (typeof alert.targetPriceInPaise === 'number' && alert.targetPriceInPaise > 0) {
      conditionMet = currentPriceInPaise <= alert.targetPriceInPaise;
      if (conditionMet) {
        reason = `Price dropped to ₹${(currentPriceInPaise / 100).toFixed(0)}, meeting your target of ₹${(alert.targetPriceInPaise / 10
      }
    }
  } else if (alert.type === 'fixed_drop') {
    const referencePrice = (typeof alert.baselinePriceInPaise === 'number' && alert.baselinePriceInPaise > 0)
      ? alert.baselinePriceInPaise
      : previousPriceInPaise;

    if (typeof referencePrice === 'number' && referencePrice > 0 && typeof alert.dropAmountInPaise === 'number') {
      const drop = referencePrice - currentPriceInPaise;
      conditionMet = drop >= alert.dropAmountInPaise;
      if (conditionMet) {
        reason = `Price dropped by ₹${(drop / 100).toFixed(0)}, meeting your drop threshold of ₹${(alert.dropAmountInPaise / 100).toFix
      }
    }
  }

  const isArmed = Boolean(alert.armed);
  const isActive = alert.status === 'active';
  const shouldNotify = Boolean(conditionMet && isArmed && isActive);
  const shouldRearm = Boolean(!conditionMet && !isArmed && isActive);
  const dedupeKey = `${alertId}:${currentPriceInPaise}`;

  return { conditionMet, shouldNotify, shouldRearm, reason, dedupeKey };
}
```

11.2 Alert Lifecycle State Machine

Event / Cycle	Current Price	Condition Met?	Armed Before	Should Notify?	Armed After	System Action
Cycle 1	₹28,000	No (> ₹25,000 target)	`true`	No	`true`	No action; alert remains armed.
Cycle 2	₹24,500	Yes ($\le$ ₹25,000 target)	`true`	**YES**	`false`	Dispatches notifications; disarms alert.
Cycle 3	₹24,500	Yes ($\le$ ₹25,000 target)	`false`	No	`false`	Duplicate suppressed; alert already disarmed.
Cycle 4	₹26,000	No (> ₹25,000 target)	`false`	No	`true`	**Re-armed automatically** for future drops.

Chapter 12 — Multi-Channel Notification Dispatch (Brevo Email & Telegram Bot)

12.1 Centralized Dispatcher (`alertDispatcher.js`)

Located in `server/src/services/alertDispatcher.js`, the dispatcher iterates through `alert.channels` (email, telegram). It checks for existing successful events in MongoDB using a composite deduplication key `` `${evalResult.dedupeKey}:${channel}` `` to prevent duplicate sends:

12.2 Notification Dispatch Sequence

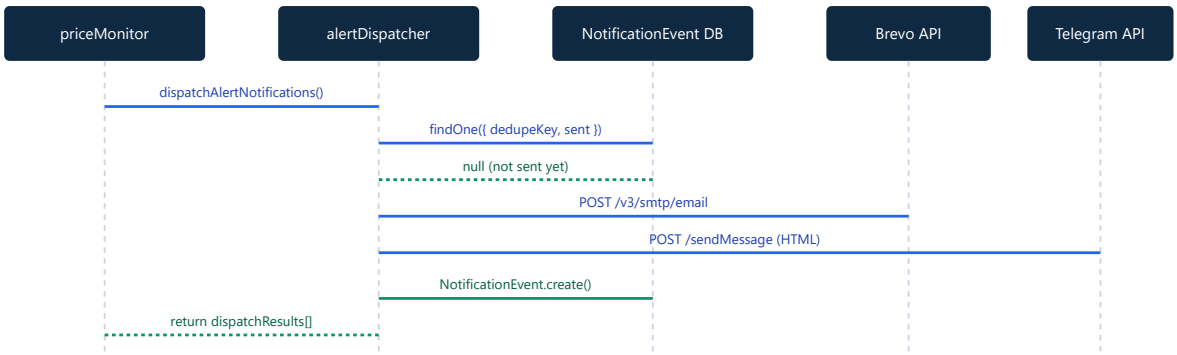


Figure 12.1: Multi-Channel Alert Dispatch & Deduplication Sequence

12.3 Partial Delivery Retry Retention

If an alert specifies both `['email', 'telegram']`, and email succeeds while Telegram fails (e.g. user has not linked Telegram yet):

- Email event is saved with `status: 'sent'`.
- Telegram event is saved with `status: 'failed'`.
- `evaluateNotificationCompletion()` determines that pending channels remain.
- The alert retains `armed: true` so the pending channel is retried during the next cycle.

Chapter 13 — Security Architecture, Authentication & Endpoint Authorization

13.1 Authentication vs. Authorization Controls

- **Authentication:** Verifies user identity via JWT (HMAC SHA-256) signed with `JWT_SECRET`. Managed by `server/src/middleware/auth.js`.
- **Ownership Authorization:** Users can only view, modify, or delete their own watchlist items, alerts, and notification history:

```
// Strict user isolation query
const alert = await Alert.findOne({ _id: req.params.alertId, userId: req.user.id });
```

- **Monitoring Trigger Secret:** `POST /api/monitor/run` requires header `X-Monitor-Secret` matching `MONITOR_TRIGGER_SECRET`. Regular JWT tokens are rejected.
- **Diagnostic Production Protection:** In production, detailed diagnostics are blocked unless the request provides `X-Diagnostic-Secret` matching `ADMIN_DIAGNOSTIC_SECRET`.

13.2 SSRF Protection in Web Extraction

Implemented in `worker/extractor.py`, the worker prevents Server-Side Request Forgery by resolving target domain IPs against a blacklist of private/internal networks before issuing HTTP requests:

```
# worker/extractor.py - SSRF Guard
DISALLOWED_NETWORKS = [
    ipaddress.ip_network("127.0.0.0/8"),      # Loopback
    ipaddress.ip_network("10.0.0.0/8"),       # Private class A
    ipaddress.ip_network("172.16.0.0/12"),    # Private class B
    ipaddress.ip_network("192.168.0.0/16"),   # Private class C
    ipaddress.ip_network("169.254.0.0/16"),   # AWS/Cloud Metadata
    ipaddress.ip_network("::1/128"),          # IPv6 loopback
]
```

Chapter 14 — CI/CD Automation, Test Containers & Cloud Deployment

14.1 Continuous Integration Pipeline (`.github/workflows/ci.yml`)

Executes on every push and pull request to `main`:

1. **Backend Tests Job:**
   - Starts an ephemeral `mongo:7.0` service container with health checks.
   - Sets up Node.js 20; installs dependencies via `npm ci`.
   - Seeds database fixtures: `npm run seed:fixtures`.
   - Runs all 25 Jest test suites: `npm test`.
2. **Frontend Build Job:**
   - Installs client dependencies; compiles Vite production bundle: `npm run build`.

14.2 Cloud Deployment Architecture

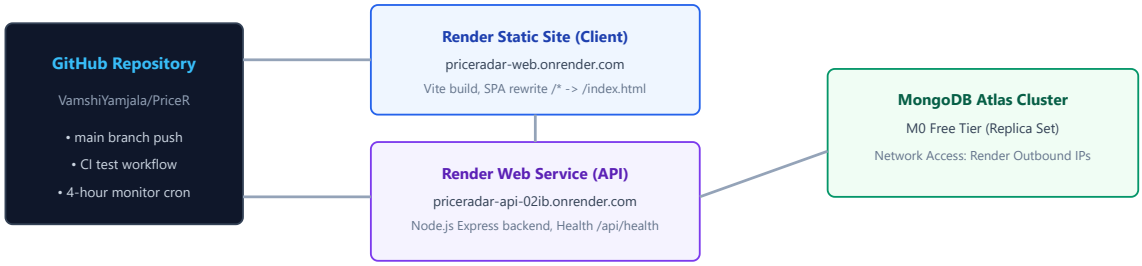


Figure 14.1: Render Cloud PaaS and MongoDB Atlas Deployment Topology

Chapter 15 — Local Development Setup, Tooling & Execution Guide

15.1 System Prerequisites

- Node.js v18.0.0 or v20.x LTS installed.
- Local MongoDB running on `mongodb://127.0.0.1:27017` or MongoDB Atlas cluster URI.
- Git installed.

15.2 Step-by-Step Setup Instructions

Step 1: Clone Repository

```
git clone https://github.com/VamshiYamjala/PriceR.git
cd PriceR
```

Step 2: Backend Setup

```
cd server
npm install
copy .env.example .env    # On Linux/macOS: cp .env.example .env
```

Configure `server/.env`:

```
PORT=5000
NODE_ENV=development
MONGODB_URI=mongodb://127.0.0.1:27017/priceradar_dev
JWT_SECRET=development-jwt-secret-at-least-32-characters-long
MONITOR_TRIGGER_SECRET=development-monitor-trigger-secret-12345
CLIENT_ORIGIN=http://localhost:5173
```

Seed initial fixtures and start development server:

```
npm run seed:fixtures
npm run dev
```

Step 3: Frontend Setup

```
# In a second terminal window
cd client
npm install
copy .env.example .env    # On Linux/macOS: cp .env.example .env
npm run dev
```

Open `http://localhost:5173` in your browser.

Chapter 16 — Testing Strategy, Open Handle Resolution & Quality Assurance

16.1 Test Suite Summary (25 Suites, 263 Tests)

The PriceRadar test suite executed on **October 20, 2026** reports **25 test suites passed, 263 tests passed, 0 failures** in ~24 seconds:

Test Suite File	Tests	Verified Coverage Scope	Teardown Safeguard
`alertEvaluator.test.js`	12	Target price & fixed drop rules, non-positive price rejection.	Unit logic (no DB).
`apifyProvider.test.js`	9	Actor input formatting, price parsing, HTTP 403 extraction.	Unit logic (no DB).
`auth.test.js`	14	Registration, login, password comparison, token verification.	`disconnectDB()`
`diagnostics.test.js`	18	Checks A through I, secret header protection, source verification.	`disconnectDB()`
`ebayProvider.test.js`	8	OAuth 2.0 client credentials, Browse API payload mapping.	Unit logic (no DB).
`emailService.test.js`	7	Brevo payload generation, sender masking, failure recovery.	Unit logic (no DB).
`health.test.js`	3	HTTP 200 database connected, HTTP 503 disconnected.	`disconnectDB()`
`models.test.js`	24	Schema validation, integer paise constraints, indexes.	`disconnectDB()`
`monitorExecution.test.js`	12	Async 202 trigger, runId polling, atomic lock concurrency.	`disconnectDB()`
`notificationFlow.test.js`	21	10 core alert conditions, sample safeguards, user isolation.	`disconnectDB()`
`priceAlertMonitoring.test.js`	18	Cycle execution, Brevo/Telegram delivery, partial failures.	`disconnectDB()`
`products.test.js`	11	Product search, categories, fallback seeder.	`disconnectDB()`
`registrationOtpFlow.test.js`	9	Step 1 OTP generation, Step 2 verification, cooldown.	`disconnectDB()`
`scheduler.test.js`	5	In-process scheduler, overlapping cycle mutex, unref timer.	Timer unref.
`watchlistAndAlerts.test.js`	14	Watchlist CRUD, immediate alert evaluation, isolation.	`disconnectDB()`
10 Other Unit Suites	78	Money math, Scrapling adapter, provider dispatcher.	Unit logic.
Total	**263**	**Full End-to-End System Integration**	**0 Open Handles**

16.2 Open Handle Teardown Resolution

Previously, tests hung in CI because `mongoose.connection.close()` did not terminate the MongoDB driver's internal topology monitoring timers. PriceRadar resolved this by exporting `disconnectDB()` in `server/src/config/db.js` using `await mongoose.disconnect()` and calling `timer.unref()` on in-process timers.

Chapter 17 — Comprehensive Troubleshooting & Failure Recovery Matrix

Symptom	Probable Cause	Diagnostic Step	Resolution
`GET /api/health` returns HTTP 503	MongoDB Atlas unreachable or Render outbound IP blocked.	Check Render logs for `Mongoose connection error`.	Add Render regional outbound CIDR blocks to Atlas Network Access list.
Monitor run reports `offersFailed: 5` with status 403	Apify account compute units expired or invalid token.	Inspect `GET /api/monitor/runs/:runId` error messages.	Update `APIFY_API_TOKEN` in Render Dashboard Environment tab.
Direct URL access shows 404 on Render frontend	Missing SPA rewrite rule on Render Static Site.	Check if reload on `/login` gives 404.	Add Rewrite Rule in Render: `/*` → `/index.html`.
GitHub Actions monitor fails with HTTP 000	Backend web service sleeping (cold start latency).	Check workflow log at wakeup ping step.	Wakeup ping (`GET /api/health`) retries automatically to allow 30-50s spin-up.
Registration OTP email never arrives	`BREVO_API_KEY` or `EMAIL_FROM` unconfigured.	Check backend logs for `[auth-otp] Verification email dispatch failed`.	Verify Brevo API key and sender email in `server/.env`.
Telegram bot doesn't link user account	Bot token missing or polling listener stopped.	Run `npm run telegram:verify` in `server/`.	Configure `TELEGRAM_BOT_TOKEN` and verify bot username.

Chapter 18 — Known Limitations & Future Architectural Roadmap

18.1 Verified Limitations

- **Apify Free Tier Compute Quota:** Live Amazon/Flipkart scraping relies on Apify cloud actors. Free accounts provide $5 monthly credit, sufficient for demonstration but requiring paid compute units for production monitoring across thousands of offers.
- **In-Process Background Memory on PaaS:** Asynchronous monitor tasks execute in-process on Render containers. While Render containers remain alive for ~15 minutes after handling requests, true crash resilience across container restarts requires an external queue (BullMQ + Redis).
- **Direct Anti-Bot Scraping Policy:** Direct DOM scraping of Amazon/Flipkart without rotating proxies is blocked by Cloudflare/PerimeterX; cloud actors remain necessary.

18.2 Future Roadmap

- BullMQ and Redis distributed message broker for horizontally scalable background workers.
- Chrome Browser Extension for one-click price tracking directly on retailer product pages.
- WhatsApp notifications via Twilio or Meta Graph API.

Chapter 19 — Complete Quick Reference

19.1 Key Repository File Paths

- **Backend Entrypoint:** `server/src/server.js`
- **Express App Core:** `server/src/app.js`
- **Price Monitor Engine:** `server/src/services/priceMonitor.js`
- **Alert Evaluator:** `server/src/services/alertEvaluator.js`
- **Alert Dispatcher:** `server/src/services/alertDispatcher.js`
- **4-Hour Monitor Workflow:** `.github/workflows/monitor.yml`
- **CI Test Workflow:** `.github/workflows/ci.yml`
- **Frontend Entrypoint:** `client/src/App.jsx`
- **Frontend API Client:** `client/src/services/api.js`

19.2 Master Command Cheat Sheet

```
# Run full backend test suite (25 suites, 263 tests)
cd server && npm test

# Run tests with open-handle detection
cd server && npm test -- --detectOpenHandles

# Seed deterministic product and offer fixtures
cd server && npm run seed:fixtures

# Verify database connection
cd server && npm run db:verify

# Build frontend production bundle
cd client && npm run build

# Start backend dev server
cd server && npm run dev

# Start frontend dev server
cd client && npm run dev
```

Chapter 20 — Project Demonstration Script

Use this structured 5–10 minute demonstration outline during project evaluations and viva examinations:

1. **Introduction (1 Min):** Introduce PriceRadar as an autonomous e-commerce price monitoring platform built on MERN, Apify, Brevo, and GitHub Actions. State the problem: shoppers miss dynamic price drops because manual tracking is tedious and laptop-dependent.
2. **User Registration & Security (1.5 Min):** Demonstrate user registration. Show the 6-digit email OTP received via Brevo. Explain password hashing with bcryptjs (10 rounds) and JWT issuance.
3. **Search & Catalog Discovery (1.5 Min):** Search for a consumer electronic item (e.g., "Sony WH-1000XM5"). Highlight how competing offers from Amazon and Flipkart appear side-by-side with lowest-price badges and integer paise savings. Explain the canonical key matching logic.
4. **Watchlist & Alert Configuration (1.5 Min):** Add a listing to the Watchlist. Create a `target_price` alert with notification channels set to Email and Telegram. Show that if the current price satisfies the target, it triggers immediately; otherwise, it arms and waits for monitoring.
5. **4-Hour Background Monitor & Distributed Lock (2 Min):** Explain the GitHub Actions 4-hour cron. Walk through the health wakeup ping, HTTP 202 asynchronous response, atomic MongoDB distributed lock (`MonitorLock`), and status polling. Emphasize that the user's computer can be completely shut down while prices are refreshed.
6. **Alert Dispatch & Conclusion (1.5 Min):** Show the email received via Brevo and mobile notification via Telegram. Demonstrate the deduplication key guarantee (anti-spam) and the re-arming lifecycle. Summarize the 25 passing test suites and production deployment on Render.